

ElasticSearch 全文检索：倒排索引表 + IK 分词器 + BM25 算法

已付费

原创 神说要有光 神光的幸福生活 2026年4月26日 13:48 山东

前面学了基于向量数据库 Milvus 实现 RAG 语义检索。

并且基于 LangGraph 实现了闭环的 Agentic RAG，也就是 Agent 自主决策要不要检索、用什么检索、信息够不够、效果怎么样、要不要重新搜。

具体的 Agentic RAG 要根据业务场景设计，理解这个闭环的思路就行。

但向量检索有个问题：

- 专业术语、精确实体更适合关键词检索，纯语义检索容易匹配不准

解决方案是：

- 同时结合关键词检索与语义检索，由模型统一融合多路结果，提升专业场景准确率

这里关键词检索用 ElasticSearch 中间件，它是专门用来实现全文检索的。

数据库是根，而中间件是特种兵。

我们会把原始数据存 MySQL，把需要检索的部分同步到 es 里。

这样关键词检索就可以走 es 了。

我们这节学一下 ElasticSearch 全文检索的中间件。

首先安装下 ES：

用上节学的 docker compose 的方式安装，之前我们安装 milvus 也是这样

创建一个目录：

```
mkdir es-test
cd es-test
npm init -y
```

添加这个 docker-compose.yml

```
version: '3.8'

services:
  # Elasticsearch 最新稳定版: 8.17.0
  es:
    image: elasticsearch:8.17.0
    container_name: es-dev
    ports:
      - "9200:9200" # ES 对外提供服务的端口
    environment:
      - discovery.type=single-node # 单节点运行 (开发环境)
      - xpack.security.enabled=false # 关闭安全认证, 免密码访问
      - xpack.security.http.ssl.enabled=false # 关闭 HTTPS 加密
      - xpack.security.transport.ssl.enabled=false # 关闭节点传输加密
      - ES_JAVA_OPTS=-Xms512m -Xmx512m # JVM 内存配置, 避免占用过高
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/es:/usr/share/elasticsearch/data
    restart: always

  # Kibana 最新稳定版: 8.17.0 (必须与 ES 版本完全一致)
  kibana:
    image: kibana:8.17.0
    container_name: kibana-dev
    ports:
      - "5601:5601" # Kibana 网页控制台端口
    environment:
      - ELASTICSEARCH_HOSTS=http://es:9200 # 连接 ES 容器内部地址
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/kibana:/usr/share/kibana/data
    restart: always
    depends_on:
```

```
- es # 等待 ES 启动完成后再启动 Kibana

networks:
  default:
    name: common-network
```

kibana 是 es 的一个可视化控制台

我们来跑一下：

神光的编程秘籍

mysql 是表和行，在 es 里就是索引和文档

es 没有 sql，都是一些 http 的接口

我们试一下索引的创建和文档的增删改查：

```
# Elasticsearch 基础操作
```

1. 查看所有索引

GET /_cat/indices?v&h=health,status,index,docs.count

2. 创建索引

PUT /article

```
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text"
      },
      "content": {
        "type": "text"
      },
      "author": {
        "type": "keyword"
      },
      "createTime": {
        "type": "date"
      },
      "viewCount": {
        "type": "integer"
      }
    }
  }
}
```

3. 查看索引结构

GET /article/_mapping

4. 查看索引配置

GET /article/_settings

5. 删除索引

DELETE /article

=====

文档 增删改查

=====

1. 新增文档 (自动生成 ID)

POST /article/_doc

```
{
  "title": "Elasticsearch 全文检索入门",
  "content": "ES 基于倒排索引与 BM25 实现全文搜索, 适用于文本检索场景",
  "author": "后端开发",
  "createTime": "2026-04-26",
  "viewCount": 128
}
```

```
}
```

```
# 2. 新增文档 (指定自定义 ID)
```

```
PUT /article/_doc/1001
```

```
{
```

```
"title": "RAG 混合检索实战",
```

```
"content": "ES 负责关键词检索, Milvus 负责向量语义检索, 结合使用效果更佳",
```

```
"author": "AI开发",
```

```
"createTime": "2026-04-26",
```

```
"viewCount": 256
```

```
}
```

```
# 3. 根据 ID 查询单条
```

```
GET /article/_doc/1001
```

```
# 4. 查询全部文档
```

```
GET /article/_search
```

```
{
```

```
"query": {
```

```
  "match_all": {}
```

```
}
```

```
}
```

```
# 5. 全文分词检索 (text 字段)
```

```
GET /article/_search
```

```
{
```

```
"query": {
```

```
  "match": {
```

```
    "content": "RAG 向量 检索"
```

```
  }
```

```
}
```

```
}
```

```
# 6. 精确匹配查询 (keyword 字段)
```

```
GET /article/_search
```

```
{
```

```
"query": {
```

```
  "term": {
```

```
    "author": "AI开发"
```

```
  }
```

```
}
```

```
}
```

```
# 7. 只返回指定字段
```

```
GET /article/_search
```

```
{
```

```
"_source": ["title", "author"],
```

```
"query": {
```

```
    "match_all": {}  
  }  
}
```

8. 分页 + 排序

```
GET /article/_search  
{  
  "from": 0,  
  "size": 10,  
  "sort": [  
    {"viewCount": "desc"}  
  ],  
  "query": {  
    "match_all": {}  
  }  
}
```

9. 局部更新文档 (推荐)

```
POST /article/_update/1001  
{  
  "doc": {  
    "viewCount": 999,  
    "title": "RAG 混合检索高级实战"  
  }  
}
```

10. 全量覆盖更新

```
PUT /article/_doc/1001  
{  
  "title": "全量覆盖测试",  
  "content": "原始内容被替换",  
  "author": "测试用户",  
  "createTime": "2026-04-26",  
  "viewCount": 66  
}
```

11. 根据 ID 删除文档

```
DELETE /article/_doc/1001
```

12. 条件批量删除

```
POST /article/_delete_by_query  
{  
  "query": {  
    "match": {  
      "author": "后端开发"  
    }  
  }  
}
```

```
# 13. 统计文档总数
GET /article/_count

# 14. 清空索引数据 (保留表结构)
POST /article/_delete_by_query
{
  "query": {
    "match_all": {}
  }
}
```

神光的编程秘籍

我们测了一遍索引的创建、文档的增删改查，整体比较简单。

那用 es 和之前用 mysql 比有啥好处呢？

ES 相比 MySQL 最大的核心优势，本质来源于倒排索引的底层设计。

普通 MySQL 使用的是正向索引：

以一行为单位存储完整数据，检索文本内容时，需要逐行遍历、逐个字段匹配内容。数据量越大、文本越长，模糊 / 全文搜索就越慢，性能极差，并不适合大范围关键词检索。

而 Elasticsearch 采用**倒排索引**机制：

会自动对 text 类型字段进行分词处理，拆解为一个个独立词条，再以「词条」为核心，反向关联所有包含该词条的文档。

简单来说：

正向索引：文档 → 关键词

倒排索引：关键词 → 文档

基于这种结构，用户输入关键词检索时，ES 只需通过词条快速匹配对应的文档，无需全表遍历，就能实现海量文本下毫秒级的全文检索。

综上，ES 倒排索引的底层架构，就是专门为海量文本、关键词模糊检索、内容匹配场景量身设计的，这也是它吊打 MySQL 全文搜索的根本原因。

理解了倒排索引，就理解了 ES 了。

显然，在 ES 里分词是很重要的，不同的分词建的索引表都不同。

es 默认的 standard 分词器对中文支持不好：

```
POST /_analyze
{
  "analyzer": "standard",
  "text": "Elasticsearch RAG 混合检索知识库"
}
```


它是每个字单独拆开的，而实际上应该“混合”、“检索”分别是整体来建立索引

这就需要用 ik 分词器了

我们安装下：

创建 elasticsearch/Dockerfile

```
# 官方 ES 基础镜像
FROM elasticsearch:8.17.0

# 安装 IK 分词（版本严格和 ES 一致）
RUN elasticsearch-plugin install --batch \
    https://release.infinilabs.com/analysis-ik/stable/elasticsearch-analysis-ik-8.17.0.zip
```

就是 es 的容器里，执行命令安装 ik 分词器插件

我们用这个 Dockerfile 来构建 es 镜像

改一下 docker-compose.yml

```
# Elasticsearch 8.17.0 + IK 中文分词（内置到镜像）
es:
  build: ./elasticsearch      # 从本地 Dockerfile 构建镜像（自带IK）
  container_name: es-dev
  ports:
    - "9200:9200"             # ES 访问端口
  environment:
    - discovery.type=single-node # 单节点运行（开发环境）
```

```
- xpack.security.enabled=false# 关闭安全认证, 免密码访问
- xpack.security.http.ssl.enabled=false# 关闭 HTTPS 加密
- xpack.security.transport.ssl.enabled=false# 关闭节点传输加密
- ES_JAVA_OPTS=-Xms512m -Xmx512m # JVM 内存配置, 避免占用过高

volumes:
  - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/es/data:/usr/share/elasticsearch/data

restart: always
```

试一下:

```
docker compose down
docker compose up -d --build
```

神光的编程秘籍

```
# 1. 检查 ES 状态
GET /

# 2. 查看已安装插件
GET /_cat/plugins?v

# 3. 原生 standard 分词
POST /_analyze
{
  "analyzer": "standard",
  "text": "Elasticsearch RAG 混合检索知识库"
}
```

```
# 4. IK 细粒度分词 (索引入库用)
POST /_analyze
{
  "analyzer": "ik_max_word",
  "text": "Elasticsearch RAG 混合检索知识库"
}

# 5. IK 智能分词 (搜索查询用)
POST /_analyze
{
  "analyzer": "ik_smart",
  "text": "Elasticsearch RAG 混合检索知识库"
}
```

有了 ik 分词器之后，就可以快速的查询关键词对应的文档了。

之前是这样的：

把搜索词分词后，去索引表匹配，然后把结果合并返回。

默认 standard 分词器拆太细，索引的意义不大。

用了 ik 分词器是这样的：

按照中文的词语来分词，建立倒排索引表。

查询的时候也用分词器分词，查询索引表，结果合并后返回。

这样的机制，自然可以毫秒级实现关键词检索。

之前创建 article 的索引用的是默认的分词器。

这次我们用 ik_max_word 来做索引生成，更细粒度（比如知识库会进一步细分为知识、库）

用 ik_smart 来做检索的分词

```
# Elasticsearch IK分词版操作手册
# 索引: life_note
# 字段全部配置 IK分词: 入库 ik_max_word / 查询 ik_smart

# 1. 查看所有索引
GET /_cat/indices?v&h=health,status,index,docs.count

# 2. 创建索引 (生活笔记场景 + IK双分词)
PUT /life_note
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "ik_max_word",
        "search_analyzer": "ik_smart"
      },
      "content": {
        "type": "text",
```

```
        "analyzer": "ik_max_word",
        "search_analyzer": "ik_smart"
    },
    "type": {
        "type": "keyword"
    },
    "author": {
        "type": "keyword"
    },
    "record_time": {
        "type": "date"
    }
}
}
```

3. 查看索引结构

GET /life_note/_mapping

4. 查看索引配置

GET /life_note/_settings

5. 删除索引

DELETE /life_note

=====

文档 增删改查 (生活日常案例)

=====

1. 新增文档 (自动生成 ID)

POST /life_note/_doc

```
{
  "title": "周末城市短途旅行攻略",
  "content": "周末适合周边短途出行，打卡公园、小吃街，放松日常工作压力，出行尽量避开早晚高峰",
  "type": "旅行生活",
  "author": "日常记录",
  "record_time": "2026-04-27"
}
```

2. 新增文档 (指定自定义 ID)

PUT /life_note/_doc/3001

```
{
  "title": "健康饮食与居家养生",
  "content": "规律作息、清淡饮食，多吃蔬菜水果，减少熬夜，合理运动才能保持身体健康",
  "type": "健康生活",
  "author": "生活达人",
  "record_time": "2026-04-27"
}
```

3. 根据 ID 查询单条

```
GET /life_note/_doc/3001
```

4. 查询全部文档

```
GET /life_note/_search
```

```
{
  "query": {
    "match_all": {}
  }
}
```

5. 全文分词检索 (IK中文分词, 搜: 健康 作息 旅行)

```
GET /life_note/_search
```

```
{
  "query": {
    "match": {
      "content": "健康 作息 旅行"
    }
  }
}
```

6. 精确匹配查询 (keyword 分类字段)

```
GET /life_note/_search
```

```
{
  "query": {
    "term": {
      "type": "健康生活"
    }
  }
}
```

7. 只返回指定字段

```
GET /life_note/_search
```

```
{
  "_source": ["title", "type", "author"],
  "query": {
    "match_all": {}
  }
}
```

8. 分页 + 时间排序

```
GET /life_note/_search
```

```
{
  "from": 0,
  "size": 10,
  "sort": [
    {"record_time": "desc"}
  ]
}
```

```
    ],  
    "query": {  
      "match_all": {}  
    }  
  }  
}
```

9. 局部更新文档 (推荐)

POST /life_note/_update/3001

```
{  
  "doc": {  
    "title": "健康饮食与居家养生小技巧",  
    "type": "居家生活"  
  }  
}
```

10. 全量覆盖更新

PUT /life_note/_doc/3001

```
{  
  "title": "日常养生好习惯总结",  
  "content": "早睡早起合理运动, 少吃油腻辛辣食物, 保持良好心态, 提升生活幸福感",  
  "type": "居家生活",  
  "author": "生活达人",  
  "record_time": "2026-04-27"  
}
```

11. 根据 ID 删除文档

DELETE /life_note/_doc/3001

12. 条件批量删除

POST /life_note/_delete_by_query

```
{  
  "query": {  
    "match": {  
      "author": "日常记录"  
    }  
  }  
}
```

13. 统计文档总数

GET /life_note/_count

14. 清空索引数据 (保留表结构)

POST /life_note/_delete_by_query

```
{  
  "query": {  
    "match_all": {}  
  }  
}
```

```
# =====  
# IK 分词测试 (生活文案)  
# =====  
  
# IK 细粒度分词 (入库存储使用)  
POST /_analyze  
{  
  "analyzer": "ik_max_word",  
  "text": "周末短途旅行 居家健康养生 日常美好生活记录"  
}  
  
# IK 智能分词 (搜索查询使用)  
POST /_analyze  
{  
  "analyzer": "ik_smart",  
  "text": "周末短途旅行 居家健康养生 日常美好生活记录"  
}
```

试一下：

神光的编程秘籍

这样，我们基于 ik 分词器实现了中文的关键词检索。

学习 ElasticSearch 还要知道一个 bm25 的算法。

虽然用 ik 分词器分词后，建立倒排索引表，然后就可以检索了。

但是检索到的文档相关性怎么样，如何排序？

这就是 bm25 算法做的。

BM25 (Best Matching 25) 是全文检索的相关性打分算法

它有一些策略，比如：

- 词频饱和：词出现次数到一定程度，分数不再涨（防堆砌）。
- 文档长度归一化：越长的文档，越“扣分”（公平对比长短文）。
- 稀有词权重高：少见词（如“IK 分词”）比常用词（如“的”）更重要。

总之，BM25 算法是公平、高效、稳定的关键词排序算法，ES 默认用它，后面 RAG 做关键词检索也是依赖这个。

理解了 ik 分词器 + 倒排索引 + BM25 算法，就理解了 Elasticsearch 了。

神光的编程秘籍

总结

这节我们学了 Elasticsearch 做全文检索。

用 docker compose 安装了 es 和它的可视化控制台 kibana

es 里只有索引、文档这两层。

我们通过 http 的接口创建了索引、文档，做了文档的增删改查。

es 的检索原理就是倒排索引，也就是关键词 -> 文档的索引表，这是它能实现海量文档毫秒级检索的核心。

入库的时候会对类型为 text 的字段分词，放到倒排索引表（类型为 keyword 的字段不会）。

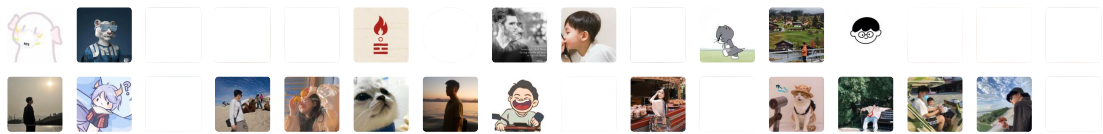
检索的时候会把 query 也分词，分别检索倒排索引表，把查到的文档合并返回。

结果还会用 BM25 算法来排序。

中文分词我们安装了 IK 分词器，入库用 ik_max_word 细粒度分词，检索用 ik_smart 高效分词。

理解了倒排索引表 + BM25 算法 + IK 分词器，串起来就理解了 Elasticsearch 的核心了。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

基于 Docker Compose 的本地开发提效和生产环境部署

下一篇

混合检索 RAG：多路召回 + 重排模型

